Assignment 5
CS 3443 Application Programming

Community Events Calendar

A community events calendar app lists upcoming local events. The main scene of the app displays brief descriptions of events (Figure 1). Each brief description consists of an event's date, category, and name. The events can be sorted by date, by category and by name. When sorting by event date, ties are broken by the event category and then the event name. When sorting by event category, ties are broken by the date and then by the name. When sorting by event name, ties are broken by date and then by category. The user can select the type of sorting by clicking on a button below the event descriptions. Next to each brief event description is a button that displays (as a second scene) the full description of the event and provides a back button to return to the event listings.

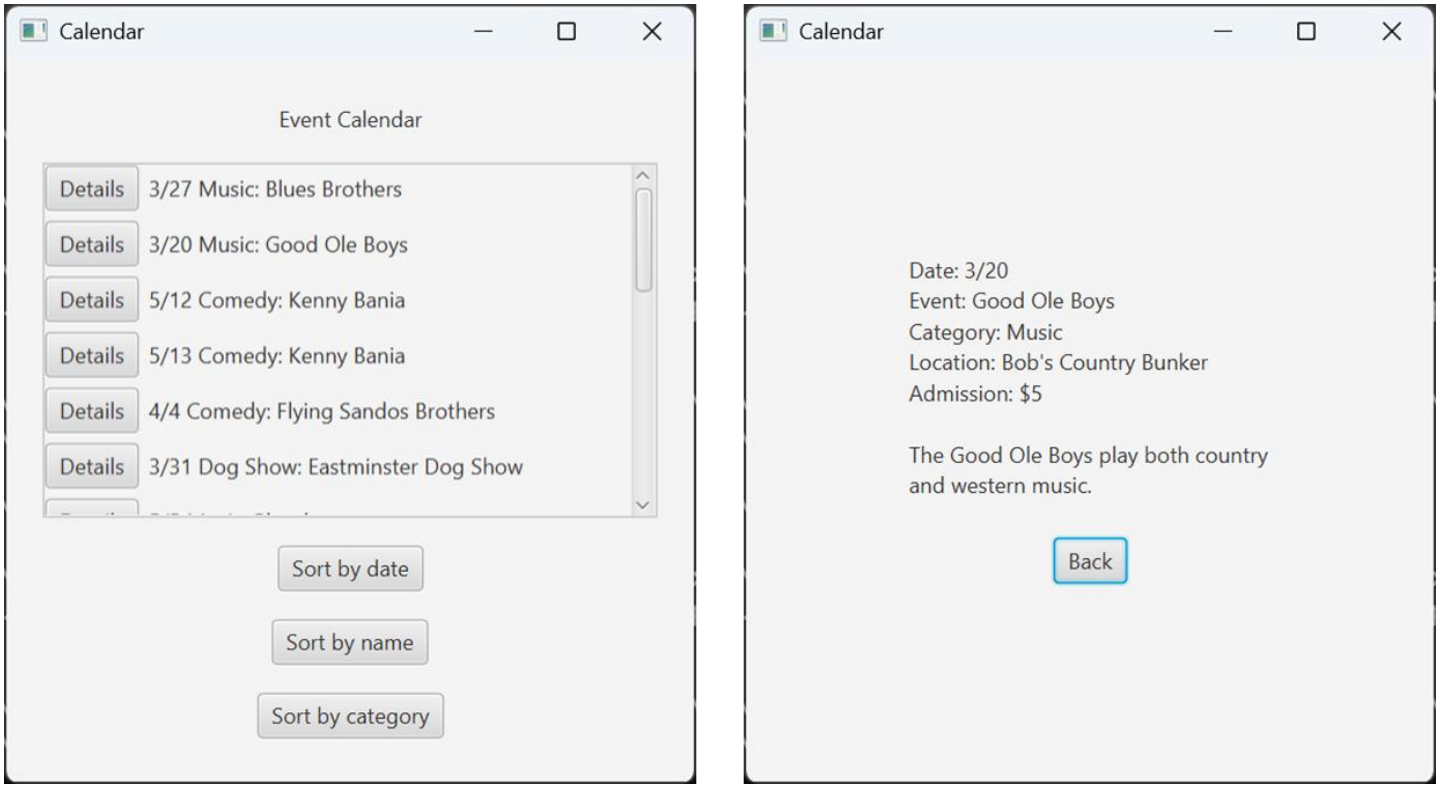

Figure 1. A community events app that can sort its events by date, category, and name (left). Clicking on the details button of an event displays detailed information about the event (right).

The event information for the app is in a comma-separated-values file containing one event per line. The order of the data on each line is: category, name, location, month, day of month, admission price, and a detailed description of the event.

Programming the Events Calendar

Create a JavaFX project named "a5_abc123" where abc123 is your UTSA ID. The project should use Java version SE 21.

Use the attached Calendar.fxml and Details.fxml files. Open them in SceneBuilder to examine the fx:ids of the labels and buttons and to examine the On Action property of the buttons. Note that the scene in Calendar.fxml has an empty VBox within a ScrollPane; this VBox has an fx:id so that it can be populated with calendar events by the application view. Do not change the fxml files.

Included with the project description is a file of comma-separated values named events.csv that contains calendar events. You may place this file anywhere in your project. Do not make any assumptions about: a) where your project might be imported to in a file system and b) what the current directory is when your program is run. This means that you should use the `getClass` and `getResource` methods of the Object and Class classes respectively to read events.csv.

Use 3 subpackages named `application.model`, `application.view`, and `application.controller`. Use the Model-View-Controller pattern and implement the following classes. You may add methods and classes.

The `application.model` package is for classes that read from a file and produce an `ArrayList` of calendar events.

- Instances of the `CalendarEvent` class represent individual events. This class must have getters for each instance variable.

- The `EventListings` class reads events from a file and produces an `ArrayList` of `CalendarEvent` objects.

The `application.view` package has classes for both scenes of the application.

- The `MainView` class is responsible for displaying the contents of the `VBox` in the `ScrollPane`.

    - The `displayEvents` method populates the initially empty `Vbox` by using the contents of the list of `CalendarEvent`s given to it by the `MainController`. This is done by creating a JavaFX `HBox` for each event and placing in it a `Button` and `Label`. For each `CalendarEvent`, do the following:

        - Create a JavaFX Details `Button` and store the `CalendarEvent` as the button's user data. The button's handler is set to be the event handler passed to the `MainView` by the `MainController`. The minimum width of the button should be set to 55 (review the `setMinWidth` method in the JavaFX Button API documentation).

        - Create a JavaFX `Label` containing a brief description of the event. Do this by passing a string consisting of the event's date (in MM/DD format), category (followed by a colon) and name to the `Label` constructor. (See the JavaFX Label api documentation). Use the `Label` class' `setMinWidth` and `setWrapText` methods to set the minimum width to 250 and to allow text wrapping.

        - Create a JavaFX `HBox` and within it place the Details `Button` and `Label`. An `HBox` constructor takes the `HBox`'s spacing (use 5), `Button` and `Label` as arguments. (See the JavaFX HBox api documentation) Place the `HBox`es into a single list (`LinkedList` or `ArrayList`) .

    Finally, after the list of `HBox`es is completed, set the `VBox`'s children to be the list of `HBox`es.

    - The `sortByCategory`, `sortByDate`, and `sortByName` methods each sort the `MainView`'s `CalendarEvent`s and then call `displayEvents`. Use the `Collections.sort` method that takes a `Comparator` as an additional argument. When sorting by category, ties are broken by the event dates (earlier events before later events). When sorting by date, ties are broken by event category (in dictionary order). When sorting by name, ties are broken by date (earlier events before later events).

- The `DetailsView` class is responsible for the Details scene's `Label` contents. It has a `displayDetails` method that takes a `CalendarEvent` as its argument and updates the `Label` by displaying the event details in the manner of Figure 2 above.

The `application.controller` package contains three classes.

- The `MainController` class includes the following methods:

- The `sortByDateAction`, `sortByCategoryAction`, and `sortByNameAction` methods are the on-click action methods for the three sorting buttons. Each of them tells the application view to sort the `CalendarEvents`.

- The `detailsScene` method takes a `CalendarEvent` as its argument, passes it to the `DetailsController`, and switches the stage's scene from the initial main scene, to the second scene that shows the details of a single event.

- The `initialize` method's duties include getting the list of `CalendarEvents` from the application model and giving it to the application view.

- The `DetailsController` class includes the following method:
  - The `backAction` method is called when the user clicks on the back button. It changes the stage's scene back to the initial scene.

  - The `initialize` method creates the `DetailsView` and passes to it the `detailsLabel`.

- The `DetailsButtonEventHandler` class implements the `EventHandler<ActionEvent>` interface by having a `handler` method that retrieves the `CalendarEvent` stored as the user data of the Details button and calls the `detailsScene` method of `MainController`.

Use the following mechanism to pass a `CalendarEvent` object to the `DetailsController`. The `MainController` gets the list of `CalendarEvents` from the application model and passes it to the `MainView`; this is done just once. When the `MainView` populates the `VBox` and creates each Details button for a `CalendarEvent`, the `CalendarEvent` is stored as user data in the button. When the user clicks on a Details button, and the `handler` method of the `DetailsButtonEventHander` class is called, the `CalendarEvent` is extracted from the button's user data and passed as an argument to the `MainController`'s `detailsScene` method which passes it to the `DetailsController` as the scene is changed.

UML Class Diagram

Use the Violet tool to create a class diagram for your implementation. It should include the classes and methods described here as well as any additional classes and/or methods that you create.

Programming Style

Use good programming style. Your comments should include a description of each class and of each method in Javadoc. Your implementation should follow the DRY principle.

Deliverables

- Create a UML class diagram of your application.
- Create a JavaFX project.
- Implement the application as described.
- Export your complete project (including events.csv) to a zip file named a5_abc123.zip where abc123 is your UTSA student ID.
- Verify that the zip file contains your work.
- Submit your zip file and UML class diagram on Canvas.
- Verify that you have submitted your zip file correctly.

Rubric

1.	Good programming style: Javadoc comments, indentation, names, DRY	10%
2.	Correctly created Model	15%
3.	Correctly created View	25%
4.	Correctly created Controller	25%
5.	All other files necessary for the app	15%

6.	UML class diagram	10%
	Total	100%

No credit is given for submissions that do not compile.